

DSA Assignment #2: 线性表 & 链表

Updated: 2026-09-16 11:05 (GMT+8)

完成学生: 张然博, 信息科学技术学院

说明:

截止日期: 前三周作业统一于 9月29日 提交至 Canvas 平台。

内容要求: 每个题目包含: **解题思路** (可选)、**源代码**、**Accepted 截图**、**预估耗时** (可选)。

本次作业对应教材 [dsa-modernization](#) 第 1~2 章 (概论、线性表)。**Python 代码必做**, **C++ 代码选做** (教材是「Python 讲算法, C++ 讲实现」, 鼓励至少挑 1~2 题用 C++ 再写一遍)。

配套讲义: [习题课: 链表 6 题](#)。建议先自己做, 卡住 20 分钟以上再看讲义。

1. 题目

E160.相交链表

hash table, linked list, two pointers, <https://leetcode.cn/problems/intersection-of-two-linked-lists/>

至少写出哈希表法; 进阶要求 $O(1)$ 空间。请在思路里写清楚双指针法为什么正确 (设两条链独有部分长 a 、 b , 公共部分长 c , 算一算两个指针各走了多少步)。

注意: 判断相交要比「是不是同一个结点」, 不是比 `val`。

思路: 先遍历一遍得到链表A和链表B的长度, 然后让长的链表先走差值步数, 这样两个指针就能同时到达相交点或者同时到达链表末尾。

预估耗时: 131ms

代码

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) ->
Optional[ListNode]:
    p1, p2 = headA, headB
    l1, l2 = 0, 0
    while p1:
        p1 = p1.next
        l1 += 1
    while p2:
        p2 = p2.next
        l2 += 1
```

```

p1, p2 = headA, headB
for _ in range(l1 - l2):
    p1 = p1.next
for _ in range(l2 - l1):
    p2 = p2.next
while p1 != p2:
    p1 = p1.next
    p2 = p2.next
return p1

```

C++代码（选做）

```

class Solution {
public:
    ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
        ListNode *p1 = headA;
        ListNode *p2 = headB;
        int l1 = 0, l2 = 0;
        while (p1) {
            p1 = p1->next;
            l1++;
        }
        while (p2) {
            p2 = p2->next;
            l2++;
        }
        p1 = headA;
        p2 = headB;
        if (l1 > l2) {
            for (int i = 0; i < l1 - l2; i++) {
                p1 = p1->next;
            }
        } else {
            for (int i = 0; i < l2 - l1; i++) {
                p2 = p2->next;
            }
        }
        while (p1 != p2) {
            p1 = p1->next;
            p2 = p2->next;
        }
        return p1;
    }
};

```

代码运行截图（至少包含有"Accepted"） 

E206.反转链表

recursion, linked list, <https://leetcode.cn/problems/reverse-linked-list/>

迭代法和递归法都要写。 递归版注意两点：`head.next.next = head` 之后必须 `head.next = None` (否则成环)；本地跑 $n = 5000$ 时要 `sys.setrecursionlimit()`。

思路：迭代法：使用三个指针，一个指向当前节点，一个指向当前节点的前一个节点，一个指向当前节点的后一个节点。遍历链表，将每个节点的next指针指向前一个节点。递归法：递归地反转链表的后半部分，然后将当前节点的next指针指向前一个节点。

预估耗时：0ms

代码（迭代）

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        pre = None
        cur = head
        while cur:
            nxt = cur.next
            cur.next = pre
            pre = cur
            cur = nxt
        return pre
```

代码（递归）

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: ListNode | None) -> ListNode | None:
        if head == None or head.next == None:
            return head
        newHead = self.reverseList(head.next)
        head.next.next = head
        head.next = None
        return newHead
```

C++代码（选做）

代码运行截图 (至少包含有"Accepted") 

M1472.设计浏览器历史记录

doubly-linked list, design, <https://leetcode.cn/problems/design-browser-history/>

用双向链表实现（这是本题的练习目的）。注意 `visit` 之后要把「前进」方向的历史切断。

思考题（写在思路里）：本题如果改用顺序表（数组 + 下标）实现，`back(steps)` / `forward(steps)` 的复杂度会变成多少？结合教材 2.4 节「不要使用链表的情况」说说哪种实现更适合这道题。

思路：使用双向链表来实现浏览器历史记录，每个节点存储一个URL，并且有指向前一个节点和后一个节点的指针。`visit` 方法会创建一个新的节点并将其添加到当前节点之后，同时切断前进方向的历史。`back` 和 `forward` 方法会根据给定的步数移动当前节点的指针，并返回当前节点的URL。改用顺序表的话，所有方法都需要数组的切片操作，复杂度为 $O(n)$ 。

预估耗时：45ms

代码

```
class Node():
    def __init__(self, url: str, next=None, prev=None):
        self.url = url
        self.next = next
        self.prev = prev

class BrowserHistory:
    def __init__(self, homepage: str):
        self.cur = Node(homepage)

    def visit(self, url: str) -> None:
        node = Node(url)
        self.cur.next = node
        node.prev = self.cur
        self.cur = node

    def back(self, steps: int) -> str:
        cnt = 0
        node = self.cur
        while cnt < steps and node.prev:
            node = node.prev
            cnt += 1
        self.cur = node
        return self.cur.url

    def forward(self, steps: int) -> str:
        cnt = 0
        node = self.cur
        while cnt < steps and node.next:
            node = node.next
            cnt += 1
        self.cur = node
```

```
return self.cur.url
```

```
# Your BrowserHistory object will be instantiated and called as such:  
# obj = BrowserHistory(homepage)  
# obj.visit(url)  
# param_2 = obj.back(steps)  
# param_3 = obj.forward(steps)
```

C++代码（选做）

代码运行截图（至少包含有"Accepted"）

M146.LRU缓存

hash table, doubly-linked list, design, <https://leetcode.cn/problems/lru-cache/>

本次作业的重点题。要求 `get` 和 `put` 均为平均 $O(1)$ ，请手写哈希表 + 双向链表，先不要直接用 `OrderedDict` / `std::list`（写完之后可以再用库实现对照一遍）。

两个最常见的坑：① 链表结点里要存 `key`，否则淘汰时删不掉哈希表里那一项；② 淘汰时链表和哈希表必须同步删。

思路：使用哈希表和双向链表来实现LRU缓存。哈希表用于快速查找缓存中的键值对，双向链表用于维护访问顺序。每次访问一个键时，将其对应的节点移动到链表的尾部，表示最近使用过。当缓存达到容量时，移除链表头部的节点，并从哈希表中删除对应的键值对。

预估耗时：31ms

代码

```
class Node:  
    def __init__(self, key: int, val: int, next=None, prev=None):  
        self.key = key  
        self.val = val  
        self.next = next  
        self.prev = prev  
  
class LRUCache:  
  
    def __init__(self, capacity: int):  
        self.hashTable = dict()  
        self.size = 0  
        self.capacity = capacity  
        self.head = None  
        self.tail = None
```

```

def moveTail(self, node: Node):
    if node == self.tail:
        return
    if node == self.head:
        self.head = node.next
        self.head.prev = None
    else:
        node.next.prev = node.prev
        node.prev.next = node.next
    node.prev = self.tail
    node.next = None
    self.tail.next = node
    self.tail = node

def get(self, key: int) -> int:
    if key in self.hashTable:
        cur = self.hashTable[key]
        self.moveTail(cur)
        return cur.val
    else:
        return -1

def put(self, key: int, value: int) -> None:
    if key in self.hashTable:
        cur = self.hashTable[key]
        cur.val = value
        self.moveTail(cur)
        return
    cur = Node(key, value, prev=self.tail)
    self.hashTable[key] = cur
    if self.head:
        self.tail.next = cur
        self.tail = cur
    else:
        self.head = cur
        self.tail = cur
    self.size += 1
    if self.size > self.capacity:
        self.size -= 1
        self.hashTable.pop(self.head.key)
        self.head = self.head.next
        self.head.prev = None

```

```

# Your LRUCache object will be instantiated and called as such:
# obj = LRUCache(capacity)
# param_1 = obj.get(key)
# obj.put(key,value)

```

C++代码 (选做)

代码运行截图 (至少包含有"Accepted") 

E21.合并两个有序链表

linked list, recursion, <https://leetcode.cn/problems/merge-two-sorted-lists/>

用**虚拟头结点 (dummy)**，体会它是怎么把「结果表还是空的」这个分支省掉的——对应教材 2.3.1 节的「头结点」。

题目要求是**拼接**已有结点，不需要 **new** 新结点。

思路：建立一个虚拟头结点，然后使用两个指针遍历两个链表，比较当前节点的值，将较小的节点连接到结果链表的尾部。继续移动指针，直到其中一个链表遍历完。最后将未遍历完的链表连接到结果链表的尾部。

预估耗时：0ms

代码

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeTwoLists(self, list1: ListNode | None, list2: ListNode | None) ->
    ListNode | None:
        dummy = ListNode()
        cur = dummy
        while list1 and list2:
            v1 = list1.val
            v2 = list2.val
            if v1 < v2:
                cur.next = list1
                list1 = list1.next
                cur = cur.next
            else:
                cur.next = list2
                list2 = list2.next
                cur = cur.next
        if list1:
            cur.next = list1
        if list2:
            cur.next = list2
        return dummy.next
```

C++代码 (选做)

代码运行截图 (至少包含有"Accepted") 

E234.回文链表

linked list, two pointers, <https://leetcode.cn/problems/palindrome-linked-list/>

进阶要求 $O(n)$ 时间、 $O(1)$ 空间：快慢指针找中点 + 反转后半段 + 逐个比较。

请在思路里说明：链表长度为奇数和偶数时，快慢指针结束后 `slow` 分别停在第几个结点上（建议用 $n = 1, 2, 3, 4$ 手推一遍）。

思路：使用快慢指针找到链表的中点，然后反转后半段链表，最后逐个比较前半段和反转后的后半段是否相等。

当链表长度为奇数时，快慢指针结束后 `slow` 停在第 $(n + 1)/2$ 个结点上；当链表长度为偶数时，快慢指针结束后 `slow` 停在第 $n/2 + 1$ 个结点上。

预估耗时：91ms

代码

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: ListNode | None) -> ListNode | None:
        if head == None or head.next == None:
            return head
        newHead = self.reverseList(head.next)
        head.next.next = head
        head.next = None
        return newHead

    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        quick, slow = head, head
        while quick and quick.next:
            quick = quick.next.next
            slow = slow.next
        newHead = self.reverseList(slow)
        cur = head
        while newHead:
            if cur.val != newHead.val:
                return False
            cur = cur.next
            newHead = newHead.next
        return True
```

C++代码（选做）

代码运行截图（至少包含有"Accepted"） 

2. 学习总结和收获

如果作业题目简单，有否额外练习题目，比如：OJ“数算2026fall每日选做”、CF、LeetCode、洛谷等网站题目。

可选的链表拓展题（讲义第6节有完整清单）：E141 环形链表、E876 链表的中间结点、M19 删除链表的倒数第N个结点、M142 环形链表 II、M2 两数相加、M148 排序链表、T25 K个一组翻转链表。

暂无